

API Design

Prepared By: Dr. D. P. Singh

API Design

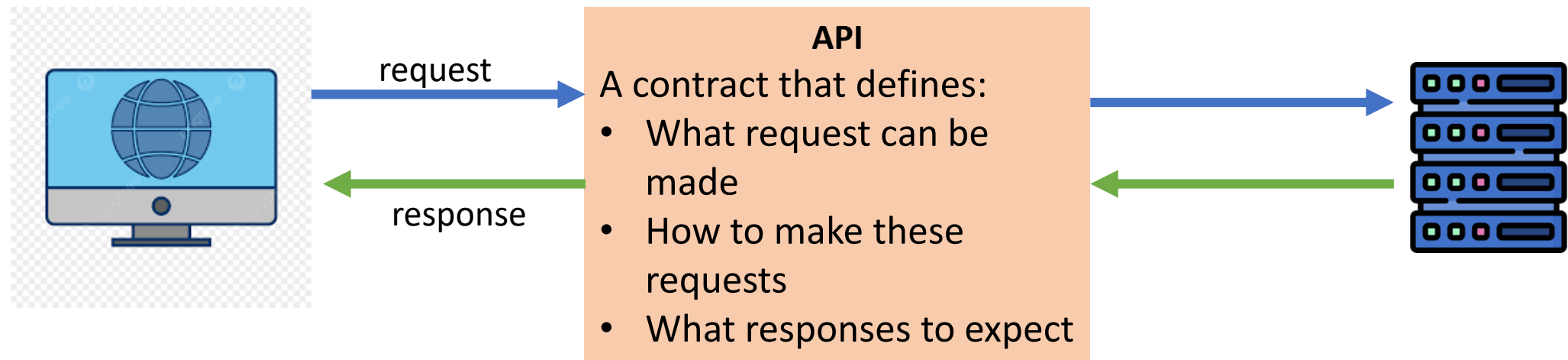
API

- Application Programming Interface define **how software components should interact**
- An Application Programming Interface (API) is a **set of rules and protocols that allows different software applications to communicate**, exchange data, and share functionality.
- Acting as a secure intermediary, APIs enable developers to integrate existing services—such as payment gateways, mapping data, or social media feeds—into their own applications without rebuilding them

API

contd...

- Act as a digital "waiter" or bridge, **receiving requests** from one app (**client**) and **returning the requested data** or action from another system (**server**). E.g. Using the Google Maps API in a food delivery app



Key Characteristics

Abstraction Mechanism → Hides implementation details while exposing functionality

Service Boundaries → Define clear interfaces between system components

Core API Styles

- REST
- GraphQL
- gRPC

Core API Styles- REST

- A REST API ([Representational State Transfer API](#)) enables communication between client and server [over HTTP](#).
- It exchanges data typically in [JSON format](#) using standard web protocols.
- Uses HTTP methods like GET, POST, PUT, PATCH, and DELETE.
- Client sends requests to server endpoints (URLs).
- Server returns responses such as JSON, XML, HTML, or images.
- Maps HTTP methods to CRUD operations (Create, Read, Update, Delete).
- Used for [Mobile and Web Apps](#)
- Most common API style

Core API Styles- GraphQL

- GraphQL is an open-source query language for APIs and a server-side runtime for executing queries.
- The GraphQL server processes **queries and returns only the requested data**.
- Unlike REST, where multiple endpoints may be required, **GraphQL retrieves specific data through a single query**.
- It was developed by **Facebook** and later open-sourced for public use.
- Used for Complex UIs
- **Example:** Suppose we have a **REST API for a blog**. If we want to get a blog post and its author information then We have to make two separate requests to the server:
 - one for the blog post.
 - another for the author's details.
- But In **GraphQL, we will request both in one query and It will reduce network overhead**.

Query:

```
query {  
  post(id: "1") {  
    title  
    content  
    author {  
      name  
      email  
    }  
  }  
}
```

Core API Styles- GraphQL

Components of GraphQL:

1. Schema

- It defines the data types that can be queried and their relationships.
- GraphQL uses its own language that is **Schema Definition Language (SDL)** for writing the schema.
- It is a human-readable language and It does not depend upon any specific language or framework. Schemas has two main types:
 - Queries (for retrieving data): Used to read and fetch data (similar to GET in REST)
 - Mutations (for modifying data): Used to modify, create, or delete data (similar to POST, PUT, or DELETE)

2. Types

- GraphQL defines custom types to **define the structure of data**. There are two main types of Type:
- **Scalar Types:** It represent values like integers, strings, booleans, and floats.
- **Object Types:** It represent complex objects with fields. Fields can be scalars or other object types. For example, **A "User" object type with fields like "id", "name", and "email"**.

Core API Styles- GraphQL

3. Queries

- It is used to retrieve data from a GraphQL server. A query specifies the fields and data that the client wants to retrieve. It is similar to GET requests in REST APIs but allow to request exactly the data we need. It is **reducing over-fetching or under-fetching**.

4. Mutations

- It is used to modify data on the server. It can be **used for creating, updating, or deleting data**. Mutations are similar to POST, PUT, or DELETE requests in REST APIs.

Core API Styles- GraphQL

- **Client Sends Query:** The client sends a query to the GraphQL server **specifying the exact data** it needs.
- **Query Validation:** The server validates the query **using the GraphQL schema** to ensure that the requested fields and types are valid.
- **Resolver Execution:** Resolvers fetch the required data from databases, APIs, or other services.
- **Data Processing:** The server collects and organizes the fetched data according to the structure of the query.
- **Response Sent to Client:** The server returns the response in **JSON format** containing only the requested data.

Core API Styles- gRPC

- gRPC Remote Procedure Calls, or gRPC for short, is an open-source **RPC framework** enabling simple and effective communication between services.
- Providing many speed advantages, **gRPC employs Protocol Buffers for serialization and HTTP/2** as its transport protocol instead of RESTful APIs using JSON over HTTP/1.1.
- gRPC allows client applications to call methods on a server application as if they were local procedure calls, simplifying the design of distributed systems.
- Used in microservices
- Provides High Performance

Protocol Buffers (Protobuf): gRPC uses Protobuf as its Interface Definition Language (IDL) and data serialization format. Protobuf is a highly efficient binary format that serializes data much faster than text-based formats like JSON. You define your service structure and APIs in a .proto file, and gRPC automatically generates idiomatic client and server code in your preferred programming language.

HTTP/2: Unlike traditional REST APIs that typically use HTTP/1.1, gRPC uses HTTP/2 as its transport protocol. This enables features like request multiplexing (sending multiple requests over a single TCP connection), binary framing, flow control, and full bi-directional streaming.

Core API Styles- gRPC

Important elements of gRPC comprise:

- gRPC helps **remote procedure calls**, therefore allowing flawless execution of functions on distant servers.
- Protocol Buffers—a **language-agnostic**, binary serialization style created by Google—define the data's structure and service approaches.
- gRPC's transport protocol, HTTP/2, **offers** capabilities including **multiplexing, flow management, and effective binary data transfer**.
- gRPC supports **several programming languages** so that services created in one language may interact with those created in another.

Core API Styles- gRPC

1. HTTP/2

- One important component of **gRPC's performance is HTTP/2 usage**. HTTP/2 brings a number of tools meant to improve communication effectiveness:
- Multiplexing—many requests and responses sent concurrently via a single TCP connection—helps to lower latency.
- **HTTP/2 compresses headers**, therefore lessening the total data sent.
- More effectively than text-based framing used in HTTP/1.1, HTTP/2 frames messages using a binary format.

2. Binary Serialisation

- Binary Serialisation Protocol Used for message **serialisation in gRPC, buffers—which enable faster parsing than conventional text-based formats like JSON or XML**—are more compact.
- Better performance results from this binary serialising style greatly shrinking the message size and the time needed for serialising and deserializing them.

Core API Styles- gRPC

3. Compression

- gRPC supports **gzip and other compression techniques** that can help to drastically cut the message size sent across the network.
- For big payloads especially, this helps since it **lessens the data transfer required**, so saving bandwidth and increasing transmission speed.

4. Streaming

- Support of streaming (unary, client, server, and bidirectional) by gRPC helps to handle big data collections and real-time communication more effectively.
- gRPC is appropriate for applications **needing real-time data processing** or significant data transfers since it allows gRPC to maximise resource use and latency by providing data as streams.

- Thank You